

Filter design techniques:

We will study some of the most popular filter design techniques:

1. Linear-phase FIR filters:

- (a) Window method
- (b) Frequency-sampled method
- (c) Weighted least-squares
- (d) Equiripple (Parks-McClellan)

2. IIR filters:

- (a) Bilinear transform
- (b) Padé's method, Prony's method, and Shank's method

$$x[n] = \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

$$x[n] * h[n] = \int_{-\pi}^{\pi} H(e^{j\omega}) X(e^{j\omega}) e^{j\omega n} d\omega = \int_{-\pi}^{\pi} A(\omega) X(e^{j\omega}) e^{j\omega(n-\ell)} d\omega$$

Generalized linear phase:

$A(\omega) = 1 \quad \forall \omega$ allpass filter

$$x[n] * h[n] = x[n-\ell]$$

- Recall that **linear phase** meant

$$H(e^{j\omega}) = A(\omega) e^{-j\omega\ell} \text{ for } A(\omega) \in \mathbb{R} \text{ and } \ell \in \mathbb{Z}.$$

- We can relax this to **generalized linear phase**:

$$H(e^{j\omega}) = A(\omega) e^{-j(\omega\ell + \phi_0)} \text{ for } 2\ell \in \mathbb{Z} \text{ and } \phi_0 \in \{0, \frac{\pi}{2}\}$$

while still retaining the desirable phase behavior.

- Interpretation: The phase derivative or *group delay*

$$g(\omega_0) \triangleq -\left. \frac{d\phi(\omega)}{d\omega} \right|_{\omega=\omega_0}$$

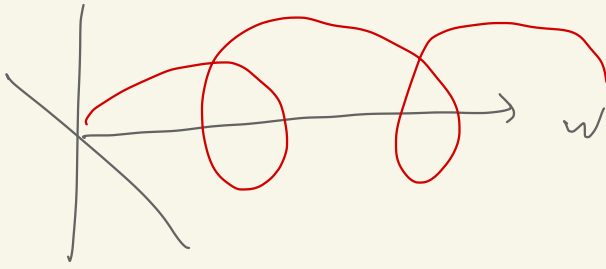
is the effective delay (in samples) caused by $H(e^{j\omega})$ at frequency ω_0 . For GLP, $\phi(\omega) = -\omega\ell - \phi_0$, and so

$$\text{GLP } H(e^{j\omega}) \text{ has } g(\omega_0) = \ell \text{ for all } \omega_0.$$

Thus, GLP filters delay all frequency components equally!

- As we will now see, the GLP property implies a form of symmetry in the FIR impulse response $h[n]$...

$$e^{j\omega}$$



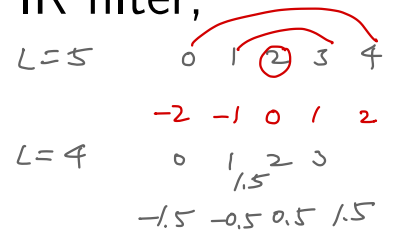
$$e^{j\theta} e^{j\omega} = e^{j(\omega+\theta)}$$

Generalized linear phase (cont.):



- Notice that, for any length- L (order $L-1$) FIR filter,

$$\begin{aligned}
 H(e^{j\omega}) &= \sum_{n=0}^{L-1} h[n] e^{-j\omega n} \\
 &= e^{-j\omega \frac{L-1}{2}} \sum_{n=0}^{L-1} h[n] e^{-j\omega (n - \frac{L-1}{2})} \\
 &= e^{-j\omega \frac{L-1}{2}} \left[h[0] e^{j\omega \frac{L-1}{2}} + h[1] e^{j\omega (\frac{L-1}{2} - 1)} \dots h[L-1] e^{-j\omega \frac{L-1}{2}} \right].
 \end{aligned}$$



Since $e^{j\theta} = \cos(\theta) + j \sin(\theta)$ and $e^{-j\theta} = \cos(\theta) - j \sin(\theta)$,

$$\begin{aligned}
 H(e^{j\omega}) &= e^{-j\omega \frac{L-1}{2}} \left[(h[0] + h[L-1]) \cos(\omega \frac{L-1}{2}) \right. \\
 &\quad + j(h[0] - h[L-1]) \sin(\omega \frac{L-1}{2}) \\
 &\quad + (h[1] + h[L-2]) \cos(\omega \frac{L-3}{2}) \\
 &\quad \left. + j(h[1] - h[L-2]) \sin(\omega \frac{L-3}{2}) + \dots \right].
 \end{aligned}$$

$A(\omega) e^{-j\phi_0}$

- To keep $A(\omega)$ in $H(e^{j\omega}) = A(\omega) e^{-j(\omega d + \phi_0)}$ real, we need:

$$\begin{aligned}
 \phi_0 = 0: & \quad a_1 + a_2 + (b_1 + b_2)j \quad b_1 + b_2 = 0 \quad h[n] = a_1 + b_1j \\
 & \quad h[L-1-n] = a_2 + b_2j \\
 \forall n: & \quad \left\{ \begin{array}{l} h[n] + h[L-1-n] \in \mathbb{R} \\ h[n] - h[L-1-n] \in \mathbb{I} \end{array} \right\} \Leftrightarrow h[n] = h^*[L-1-n]
 \end{aligned}$$

$$\begin{aligned}
 \phi_0 = \frac{\pi}{2}: & \quad a_1 - a_2 + (b_1 - b_2)j \in \mathbb{I} \quad a_1 - a_2 = 0 \\
 & \quad \text{conjugate symmetry around } \frac{L-1}{2}. \\
 \forall n: & \quad \left\{ \begin{array}{l} h[n] + h[L-1-n] \in \mathbb{I} \\ h[n] - h[L-1-n] \in \mathbb{R} \end{array} \right\} \Leftrightarrow h[n] = -h^*[L-1-n] \\
 & \quad \text{conjugate anti-symmetry around } \frac{L-1}{2}.
 \end{aligned}$$

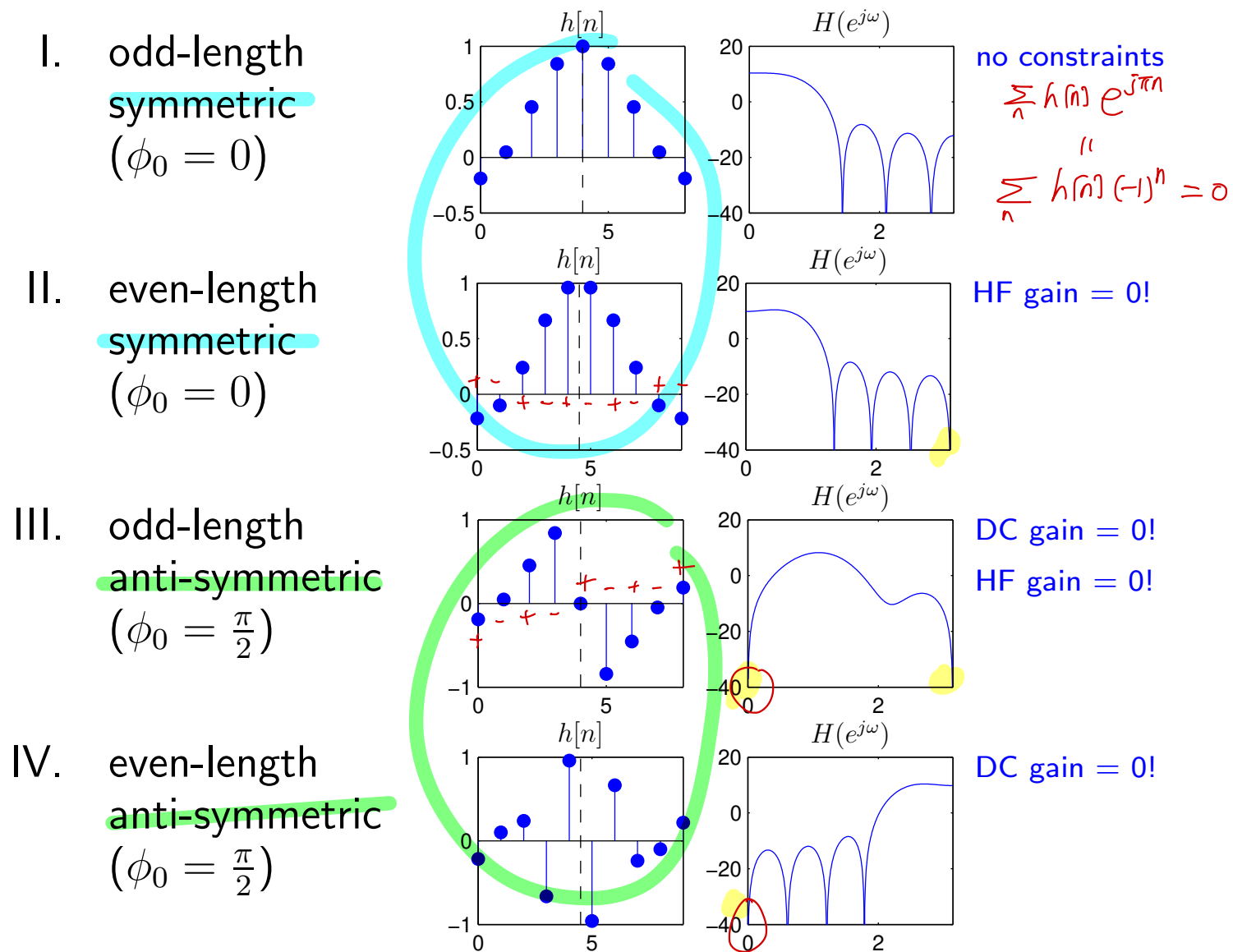
- Often we're interested in real-valued $h[n]$. In this case, GLP implies *symmetry* or *anti-symmetry* around the point $\frac{L-1}{2}$.

Generalized linear phase (cont.):

There are some implications to symmetry and anti-symmetry.

Consider that $\begin{cases} \text{DC gain: } H(e^{j0}) = \sum_{n=0}^{L-1} h[n], \\ \text{HF gain: } H(e^{j\pi}) = \sum_{n=0}^{L-1} h[n](-1)^n. \end{cases}$

As a consequence, we have four “types” of *real*-GLP filter:



Note: group delay = $\frac{L-1}{2}$ whether length L is even or odd.

Window-based FIR design:

To design a (causal) length- L GLP filter $h[n]$,

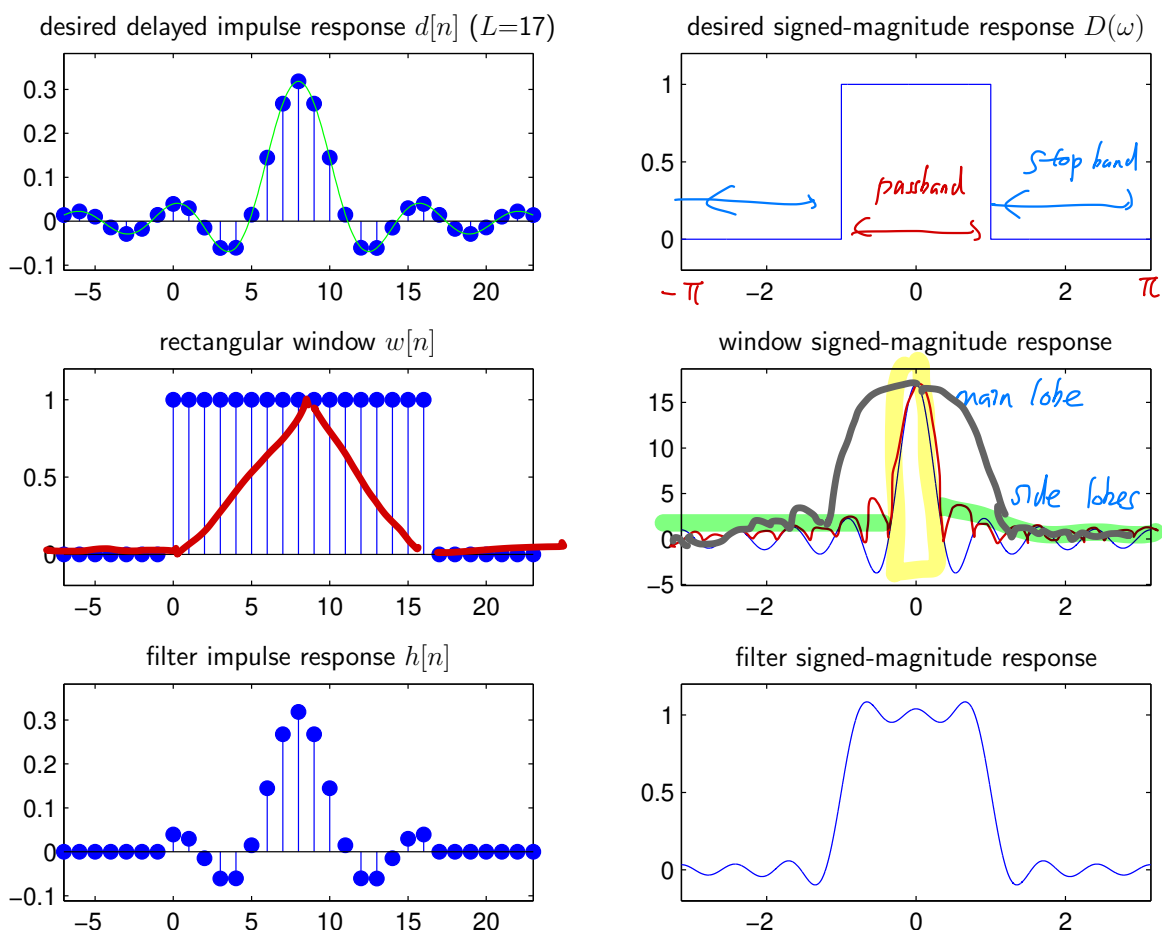
1. Specify desired signed-magnitude response $D(\omega) \in \mathbb{R}$,
2. Derive desired $\frac{L-1}{2}$ -delayed impulse response:

$$d[n] = \mathcal{F}_{\text{DTFT}}^{-1} \{ D(\omega) e^{-j(\omega \frac{L-1}{2} + \phi_0)} \}, \quad n \in \mathbb{Z}$$

3. Apply causal length- L window $w[n]$ to $d[n]$:

$$h[n] = w[n]d[n].$$

Recall: time-domain windowing $\Leftrightarrow$ freq-domain convolution.



Window-based FIR design (cont.):

Through choice of window, we can trade off between:

1. transition-band width $\Delta_\omega = |\omega_p - \omega_s|$ (in rad/sample)
2. passband/stopband deviation (Note: $\delta_p = \delta_s$ here!)

Examples for several windows:

window	$\Delta_\omega/2\pi$	$\delta_p = \delta_s$	A_p dB	A_s dB
rectangular	$0.9/L$	0.09	0.82	21
Hann	$3.1/L$	0.0063	0.055	44
Hamming	$3.3/L$	0.0022	0.019	53
Blackman	$5.5/L$	0.0002	0.0017	74

The **Kaiser window** gives a direct way to attain a given A_s through the choice of its window parameter β :

$$\beta = \begin{cases} 0 & A_s \leq 21 \\ 0.5842(A_s - 21)^{0.4} + 0.07886(A_s - 21) & 21 < A_s \leq 50 \\ 0.1102(A_s - 8.7) & 50 < A_s \end{cases}$$

and to attain a given (A_s, Δ_ω) through choice of length L :

$$L \approx \frac{A_s - 7.95}{2.285\Delta_\omega} + 1.$$

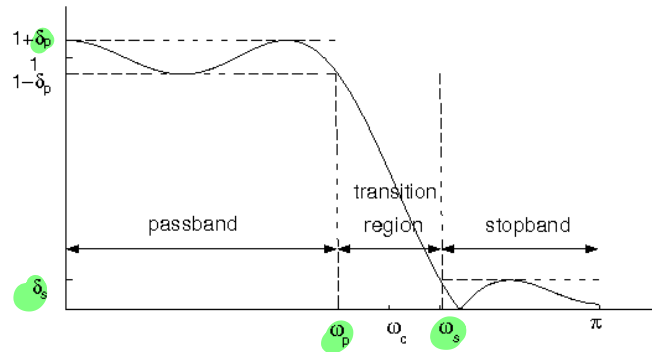
Selection of (β, L) is automated by Matlab's `kaiserord`.

To optimize window designs, a bit of tweaking may be needed!

Example of window-based FIR design:

- Say that we desire a linear-phase LPF with

$$\omega_p = 0.2\pi, \omega_s = 0.3\pi, \delta_p \leq 0.02, \delta_s \leq 0.01.$$



- For window-based filters, $\delta_p = \delta_s$. We choose $\delta_p = \delta_s = 0.01$, or $A_s = -20 \log_{10}(0.01) = 40$ dB, to meet both δ constraints.
- Based on the δ values in the table, the rectangular window won't work, but any of the others will. Of those, the Hann window has sharpest transition. Kaiser would also work...
- For Hann, $\frac{\Delta\omega}{2\pi} = \frac{3.1}{L}$. Since $\Delta\omega = 0.1\pi$, we need $L \approx 62$.
For Matlab, we use the "cutoff" freq $\omega_c = \frac{\omega_p + \omega_s}{2}$ and type

```
h = fir1(L-1, ωc/π, hann(L), 'noscale');
```

- $L-1$ specifies the filter *order*
- ω_c/π is the "Matlab-normalized" cutoff frequency
(a Matlab frequency of "1" = Nyquist = π rad/samp)
- 'noscale' disables scaling of passband midpoint to 0 dB.

- For Kaiser, we can use Matlab's automated design procedure:

```
[N,fc,beta,type]=kaiserord([0.2,0.3],[1,0],[0.02,0.01]);  
h=fir1(N,fc,type,kaiser(N+1,beta),'noscale');
```


Example of window-based FIR design (cont.):

The following code designs a filter and plots its DTFT magnitude, zooming into the passband and stopband edges.

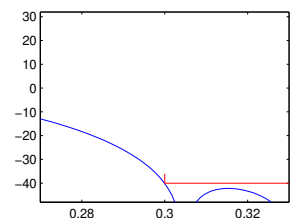
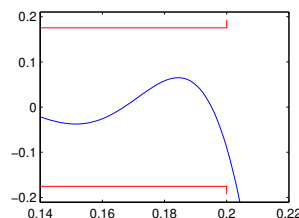
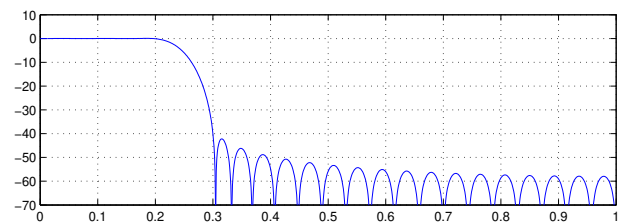
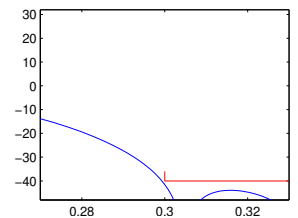
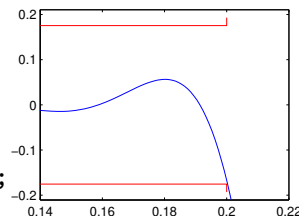
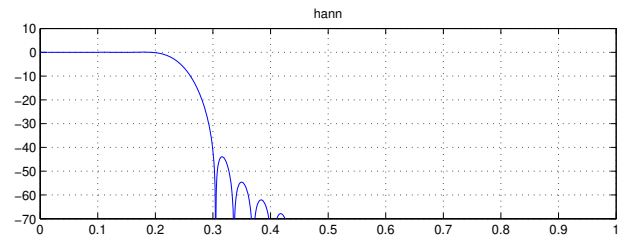
```
fp = 0.2; % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

Ap = -20*log10(1-delp);
As = -20*log10(dels);
delm = min(delp,dels);
Asm = -20*log10(delm);
fc = (fp+fs)/2;

L=60;
h=fir1(L-1,0.992*fc,hann(L));

%[N,fc,beta,type]=kaiserord([0.2,0.3],[1,0],[0.02,0.01]);
%h=fir1(N,fc,type,kaiser(N+1,beta),'noscale');

[H,omega]=freqz(h,1,1024);
f = omega/pi;
subplot(2,1,1);
plot(f,20*log10(abs(H)));
grid on;
axis([0,1,-70,10])
subplot(2,2,3);
plot(f,20*log10(abs(H)),...
     [0,fp,fp],-Ap*[1,1,1.1],'r',...
     [0,fp,fp],Ap*[1,1,1.1],'r');
axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
plot(f,20*log10(abs(H)),...
     [fs,fs,1],-As*[0.9,1,1],'r');
axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```



For Hann (but not Kaiser), ω_c was tweaked & L minimized!

Frequency-sampled FIR design:

- Given desired signed-magnitude response $D(\omega)$, it's possible to design length- L GLP $h[n]$ that *exactly* equals the desired frequency response at L frequencies $\{\omega_k\}_{k=0}^{L-1}$.
- For a length- L GLP filter, the desired freq response is

$$D(e^{j\omega}) = D(\omega)e^{-j(\omega\frac{L-1}{2}+\phi_0)},$$

and so matching it at frequencies $\{\omega_k\}_{k=0}^{L-1}$ means

$$D(e^{j\omega_k}) = H(e^{j\omega_k}) = \sum_{n=0}^{L-1} h[n]e^{-j\omega_k n} \text{ for } k = 0 \dots L-1.$$

This gives a system of L linear equations in L unknowns:

$$\underbrace{\begin{bmatrix} D(e^{j\omega_0}) \\ \vdots \\ D(e^{j\omega_{L-1}}) \end{bmatrix}}_{\mathbf{d}} = \underbrace{\begin{bmatrix} e^{-j\omega_0 0} & \dots & e^{-j\omega_0(L-1)} \\ \vdots & \ddots & \vdots \\ e^{-j\omega_{L-1} 0} & \dots & e^{-j\omega_{L-1}(L-1)} \end{bmatrix}}_{\mathbf{W}} \underbrace{\begin{bmatrix} h[0] \\ \vdots \\ h[L-1] \end{bmatrix}}_{\mathbf{h}}$$

which can be solved via:

$$\mathbf{h} = \mathbf{W}^{-1}\mathbf{d}.$$

- If the frequencies are chosen such that $\omega_k = \frac{2\pi}{L}k$, we get

$$\mathbf{h} = \mathbf{W}_{L\text{-DFT}}^{-1}\mathbf{d}$$

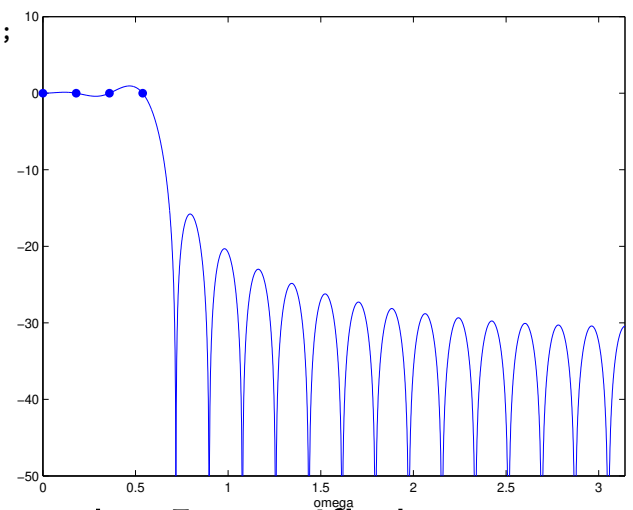
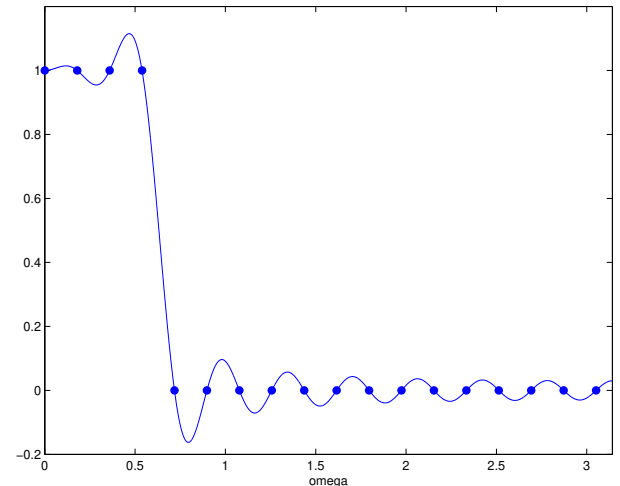
which can be calculated efficiently using an L -IFFT.

Example of frequency-sampled FIR design:

The following code designs a length-35 GLP LPF with cutoff frequency $\omega_c = \frac{2\pi}{10}$, and plots both linear signed-magnitude response and dB-magnitude response.

```
L = 35;
phi0 = 0;
omega_c = 0.1*2*pi;
omega = 2*pi*[0:L-1]/L;
pass1 = find(omega<omega_c);
pass2 = find(omega>2*pi-omega_c);
D = zeros(1,L);
D(pass1) = exp(-j*((L-1)/2*omega(pass1)+phi0));
D(pass2) = exp(-j*((L-1)/2*omega(pass2)+phi0));
h = real(ifft(D));

M = 8192;
H = real(fft(h,M).*exp(j*((L-1)/2*[0:M-1]/M*2*pi+phi0)));
figure(1); clf;
plot([0:M-1]/M*2*pi,H,[0:L-1]/L*2*pi,abs(D),'b');
axis([0,pi,-0.2,1.2])
xlabel('omega')
figure(2); clf;
plot([0:M-1]/M*2*pi,20*log10(abs(H)),...
     [0:L-1]/L*2*pi,20*log10(abs(D)),'b');
axis([0,pi,-50,10])
xlabel('omega')
```



While the response is indeed perfect at the L specified frequencies, it is not very good inbetween!

Performance can be improved by windowing the resulting impulse response, as automated by Matlab's `fir2` ...

Example of fir2-based FIR design:

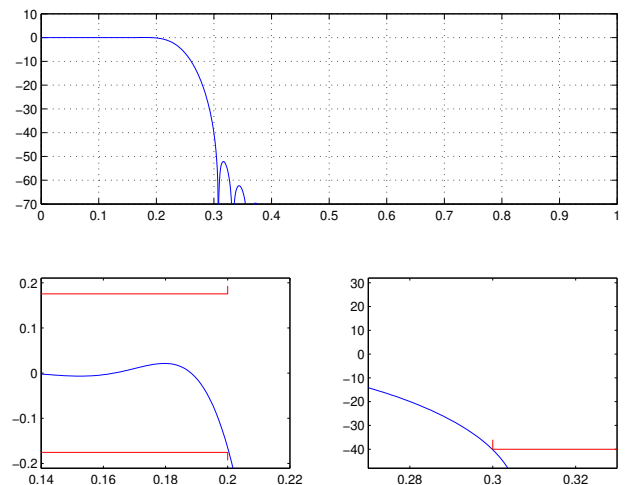
`fir2` performs frequency-sampled FIR design with automatic impulse-response windowing (default is Hamming).

Let's try `fir2` on the same LPF design considered earlier:

```
fp = 0.2; % matlab-normalized frequency!
fs = 0.3;
delp = 0.02;
dels = 0.01;

fc = (fp+fs)/2;
L=68;
h=fir2(L-1,[0,0.991*fc,0.991*fc,1],[1,1,0,0],hann(L));
```

```
Ap = -20*log10(1-delp);
As = -20*log10(dels);
[H,omega]=freqz(h,1,1024);
f = omega/pi;
subplot(2,1,1);
plot(f,20*log10(abs(H)));
grid on;
axis([0,1,-70,10])
subplot(2,2,3);
plot(f,20*log10(abs(H)),...
     [0,fp,fp],-Ap*[1,1,1,1],'r',...
     [0,fp,fp],Ap*[1,1,1,1],'r');
axis([0.7*fp,1.1*fp,-1.2*Ap,1.2*Ap]);
subplot(2,2,4);
plot(f,20*log10(abs(H)),...
     [fs,fs,1],-As*[0.9,1,1],'r');
axis([0.9*fs,1.1*fs,-1.2*As,0.8*As]);
```



Even after tweaking, `fir2` requires a longer filter! :(

The advantage to `fir2` is that it works with *any* $D(e^{j\omega})$; no complicated steps like $\mathcal{F}_{\text{DTFT}}^{-1}\{D(e^{j\omega})\}$ are needed!